



Web Programming

MongoDB
Lecture 17 & 18
Mahhek Tahir



Intro to MongoDB

MongoDB is a **NoSQL**, **document-oriented database**.

Think of **MongoDB** like a giant **online cupboard** where you store your app's information like users, posts, comments but instead of folders and files, you use **collections and documents**.

It stores data in a **flexible format** (like JSON).

You don't have to define all the fields beforehand like you do in SQL (MySQL/PostgreSQL).



Example

```
{  
  "_id": "123",  
  "name": "Ali",  
  "age": 25,  
  "skills": ["JavaScript", "Node.js"]  
}
```

Mongoose



Mongoose is like a **translator** between your JavaScript code and the MongoDB database.

- Helps you define **rules** for your data (like "age must be a number").
- Makes it easier to **connect**, **save**, and **fetch** data.
- You define "blueprints" for your data using something called a **Schema**.



Schema in Mongoose

- ▶ A **schema** is like a **blueprint** of your data. You tell Mongoose:
“Hey, every user must have a name (as a string), and an age (as a number).”
- ▶ Example:

```
const userSchema = new mongoose.Schema({  
  name: String,  
  age: Number  
});
```

This means:

name should be text

age should be a number



MongoDB DataType

Here are the **most common types** of data you'll use:

Type	Description
String	Text values
Number	Numeric values
Boolean	true / false
Array	List of values
ObjectId	Reference to another MongoDB doc
Date	Date and time values



Validation

- `required: true` → Field must be filled
- `min: 18` → Minimum value `maxLength: 50` → Max characters
- `match: /pattern/` → Match a regex pattern

```
const userSchema = new mongoose.Schema({  
  name: { type: String, required: true },  
  email: { type: String, match: /@/, required: true },  
  age: { type: Number, min: 18 },  
  isActive: { type: Boolean, default: true }  
});
```

User Schema



```
const userSchema = new mongoose.Schema({  
  name: {type: String, required: true, minLength: 3},  
  email: {type: String, match: /.+\@.+\..+/,  
    required: true},  
  age: {type: Number, min: 18,max: 60},  
  isActive: {type: Boolean,default: true}  
});
```




Common schema options

```
age: { type: Number, default: 18 }
email: { type: String, unique: true }
username: { type: String, index: true }
status: { type: String, enum: ['active', 'inactive'] }
age: { type: Number, min: 0, max: 120 }
name: { type: String, minlength: 3, maxlength: 50 }
email: { type: String, match: /.+\@.+\..+/ }
createdAt: { type: Date, immutable: true } (prevents updation after creation)
first_name: { type: String, alias: 'firstName' }
name: {
  type: String,
  set: v => v.trim()
}
password: { type: String, select: false }
```



Select: false

By default, this field will NOT be returned when you fetch data from the database

```
const userSchema = new mongoose.Schema({  
  email: String,  
  password: { type: String, select: false }  
});
```

```
const user = await User.findOne({ email: "test@gmail.com" });  
console.log(user);
```

Mainly for **security**

```
const user = await User.findOne({ email }).select('+password');
```



min & minLength

min: It sets the **minimum value** a number field is allowed to have.

minLength: It sets the **minimum number of characters** in a string.

```
const userSchema = new mongoose.Schema({  
  age: { type: Number, min: 18 }  
  name: { type: String, minLength: 3 }  
});
```

This means **age** must be **at least 18**.

This means **name** must be **at least 3 characters** long.



Timestamp

Mongoose can **automatically track** when your data is created and updated.

```
const postSchema = new mongoose.Schema({  
  title: String,  
  content: String  
}, { timestamps: true });
```

You'll now get:

- **createdAt**: When the document was made
- **updatedAt**: When it was last changed



Defaults

In Mongoose, you can set default values for fields when **documents are created**. This is useful when certain fields **should always have a value** if not provided by the user.

```
const userSchema = new mongoose.Schema({  
  name: { type: String, required: true },  
  age: { type: Number, default: 18 },  
  isActive: { type: Boolean, default: true }  
});
```

`age` will default to **18** if no value is provided.

`isActive` will default to **true** if not specified.

Timestamps: Automatically set the `createdAt` and `updatedAt` dates



Relations

MongoDB is a NoSQL database, so it's **document-based**, not table-based.

But we still need to **connect data across collections**, like:

- A **User** has many **Posts**
- A **Post** has many **Comments**
- A **Product** has many **Reviews**

These connections are called **relationships between documents**



1-to-1 Relationship

```
const userSchema = new mongoose.Schema({  
  name: { type: String, required: true },  
  email: { type: String, required: true, unique: true }  
});
```

```
userSchema.virtual("profile", {ref: "Profile", localField: "_id",  
  foreignField: "user", justOne: true});
```

```
userSchema.set("toJSON", { virtuals: true });
```

```
const profileSchema = new mongoose.Schema({bio: String,  
  location: String,  
  user: {type: mongoose.Schema.Types.ObjectId, ref: "User", required:  
    true, unique: true}});
```



User:

```
{  
  "_id": ObjectId("661191abcde1234567890001"),  
  "name": "John Ibrahim",  
  "email": "john@example.com",  
}
```

Profile:

```
{  
  "_id": ObjectId("661191fghij1234567890002"),  
  "bio": "Full-stack developer and tech enthusiast.",  
  "location": "San Francisco",  
  "user": ObjectId("661191abcde1234567890001")  
}
```




Populate()

The `populate()` method in Mongoose is used to **populate** references to documents from other collections. It allows you to perform **joins** (similar to SQL joins) in MongoDB by fetching documents from other collections based on referenced `ObjectId` fields.



```
const user = await User.findById(userId).populate("profile");
```

```
{
  "_id": "user1",
  "name": "John",
  "email": "john@example.com",
  "profile": {
    "_id": "profile1",
    "bio": "Developer",
    "user": "user1"
  }
}
```

```
const profile = await Profile.findOne({ user: userId }).populate("user");
```

```
{
  "_id": "profile1",
  "bio": "Developer",
  "user": {
    "_id": "user1",
    "name": "John"
  }
}
```



1 to Many

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true
}
});

const postSchema = new mongoose.Schema({
  title: { type: String, required: true },
  content: { type: String, required: true },
  user: { type: mongoose.Schema.Types.ObjectId, ref:
'User', required: true } // Reference to User
});
```



User:

```
{
  "_id": ObjectId("66118f84d2e4f32ff7a12345"),
  "name": "Alice",
  "email": "alice@example.com"
}
```

Post:

```
{
  "_id": ObjectId("66119021d2e4f32ff7a67890"),
  "title": "My First Blog",
  "content": "This is my very first blog post using
Mongoose!",
  "user": ObjectId("66118f84d2e4f32ff7a12345")  // Points
to Alice
}
```



```
const posts = await Post.find().populate("user");
```

```
[
  {
    "_id": "post1",
    "title": "Hello",
    "user": {
      "_id": "user1",
      "name": "John"
    }
  }
]
```

```
const user = await User.findById(userId).populate("posts");
```

```
{
  "_id": "user1",
  "name": "John",
  "posts": [
    {
      "_id": "post1",
      "title": "Hello"
    },
    {
      "_id": "post2",
      "title": "Another Post"
    }
  ]
}
```



Many to Many

```
const studentSchema = new mongoose.Schema({  
  name: { type: String, required: true },  
  courses: [{ type: mongoose.Schema.Types.ObjectId,  
    ref: "Course" }]});
```

```
const courseSchema = new mongoose.Schema({  
  title: { type: String, required: true }});  
//Virtual  
courseSchema.virtual("students", {ref: "Student",  
  localField: "_id", foreignField: "courses"  
});
```

Student:

Student:

```
{  
  "_id": "student1",  
  "name": "Ali",  
  "courses": ["course1", "course2"]  
}
```

Courses:

```
{  
  "_id":  
  ObjectId("course1"),  
  "title": "Math"  
}
```

```
{  
  "_id":  
  ObjectId("course2"),  
  "title": "Physics"  
}
```





```
const student = await
Student.findById(studentId).popula
te("courses");
```

```
{
  "_id": "student1",
  "name": "Ali",
  "courses": [
    {
      "_id": "course1",
      "title": "Math"
    },
    {
      "_id": "course2",
      "title": "Physics"
    }
  ]
}
```

```
const course = await
Course.findById(courseId).populate("students");
```

```
{
  "_id": "course1",
  "title": "Math",
  "students": [
    {
      "_id": "student1",
      "name": "Ali"
    }
  ]
}
```




Defining a Schema and Model (.js file)

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});

const User = mongoose.model('User',
userSchema);
```

Schema defines the structure

Model is used to interact with the collection

C - Create



```
try {  
  const user = await User.create({  
    name: 'Ali',  
    email: 'ali@example.com',  
    age: 25  
  });  
  
  console.log("Saved:", user);  
} catch (err) {  
  console.error("Error:", err);  
}
```



R - Read

```
// Find All Users
```

```
User.find()  
  .then(users => console.log(users));
```

```
//Find by ID
```

```
User.findById('ID_HERE')  
  .then(user => console.log(user));
```

```
//Find One with Condition
```

```
User.findOne({ email: 'john@example.com' })  
  .then(user => console.log(user))  
  .catch(err => console.log('Error:', err));
```



R - Read

```
//Find Multiple with condition
```

```
User.find({ name: 'John Doe' })  
  .then(users => console.log(users))  
  .catch(err => console.log('Error:', err));
```

```
//Find Multiple with multiple condition (AND)
```

```
User.find({  
  age: { $gte: 18 }, // Age greater than or equal to 18  
  name: 'John Doe' // Name should be 'John Doe'  
})  
  .then(users => console.log(users))  
  .catch(err => console.log('Error:', err));
```



R - READ

//Find with Multiple Conditions (OR)

```
User.find({
  $or: [
    { age: { $gte: 18 } },
    { name: 'John Doe' }
  ]
})
.then(users => console.log(users))
.catch(err => console.log('Error:', err));
```



READ

```
User.find({ age: { $ne: 30 } }) // $ne: Not equal to  
User.find({ age: { $gt: 18 } }) // $gt: Greater than  
User.find({ age: { $gte: 18 } }) // $gte: Greater than or  
equal to
```

```
User.find({ age: { $lt: 30 } }) $lt: Less than  
User.find({ age: { $lte: 30 } }) $lte: Less than or equal to
```

```
User.find({ age: { $in: [20, 25, 30] } }) $in  
User.find({ age: { $nin: [20, 25, 30] } }) $nin: Not in
```



READ with async/await

```
async function doStuff() {  
  const users = await User.find();  
  console.log(users);  
}
```



Sort

```
User.find({ name: "John" }).sort({ age: 1 }) //  
Youngest John first  
User.find({ name: "John" }).sort({ createdAt: -1  
}) // Most recent John  
User.find().sort({ score: -1, username: -1 });  
//Multiple sort
```

positive → ascending

negative → descending



U - Update

To update an existing product, use the `findByIdAndUpdate()` method:

```
const userId = 'user_id_here';
```

```
user =
```

```
User.findByIdAndUpdate(userId, {  
  age: 30 }, { new: true })
```

`{ new: true }` returns updated document
else old document



U - Update

// Update a single user named 'John'

```
User.findOneAndUpdate({ name: 'john' }, { age: 30 }, { new: true })
```

The first document it finds based on its internal execution plan, if you want to be sure use sort

```
User.findOne({ name: "john" }).sort({ _id: 1 })
```

*// To update **all users** named 'John', you would use **updateMany()***

```
User.updateMany({ name: 'john' }, { age: 30 })
```



Update and validate

Mongoose **does NOT run validations** on update queries because MongoDB runs updates directly on the database — it skips the Mongoose document layer where validation happens.

```
User.findByIdAndUpdate(userId, { age:
30 }, { new: true, runValidators: true
})
  .then(user => console.log('Updated:',
user));
```



Multiple updates

```
userID = "whatever"
User.findByIdAndUpdate(
  userID,
  { age: 30, email: 'newemail@example.com' },
  { new: true })
.then(user => console.log('Updated:', user))
```



D - Delete

```
User.findByIdAndDelete('ID_HERE')  
  .then(() => console.log('User deleted'));
```

Also supports `.deleteOne()` or `.deleteMany()`

CRUD with Relation (1-M)



We'll create a **User** model and a **Post** model. A user can have many posts, so we will establish a one-to-many relationship between **User** and **Post** where:

- A user can have multiple posts.
- Each post belongs to a single user.

CREATE



```
const mongoose = require('mongoose');
const User = require('./models/User');
const Post = require('./models/Post');
const createUserAndPost = async () => {
  const user = new User({
    name: 'Alice',
    email: 'alice@example.com'
  });
  await user.save(); // Save the user
  // Create a post for this user
  const post = new Post({
    title: 'First Post',
    content: 'This is my first post.',
    user: user._id // Reference the user. why _id?
  });
```

id vs _id



`_id`:

- **Type:** `ObjectId`
- This is the **actual identifier**
`ObjectId("603d5f4e6e1d2b1e44a9bbd9")`.
- You typically see `_id` when you query documents directly in MongoDB or Mongoose. It's a **unique identifier** for each document.

2. `id`:

- **Type:** `String`
- **String representation** of the `_id` field.
- `ObjectId("603d5f4e6e1d2b1e44a9bbd9")`, the `id` would return `"603d5f4e6e1d2b1e44a9bbd9"`, which is easier to work with in some cases.
- The `id` property is especially useful when you need to send the ID as a string in API responses or in JavaScript code.

Specifying Which Fields to Populate



```
const posts = await Post.find()  
  .populate('user', 'name email')           // post.user  
  .populate('comments');                    // post.comments
```



Resources

MongoDB (General)

1. MongoDB University (Official & Free!)

- **Website:** university.mongodb.com
- **Courses:**
 - MongoDB for Developers (Beginner/Intermediate)
 - MongoDB Aggregation Framework (Advanced)

2. MongoDB Manual (Official Docs)

- **Website:** <https://www.mongodb.com/docs/manual/>

3. YouTube Channel

- **The Net Ninja – MongoDB Tutorial for Beginners**
[Watch on YouTube](#)
Very beginner-friendly with clear explanations.
- **Academind – MongoDB Crash Course**
[Watch here](#)



Mongoose (ODM for MongoDB with Node.js)

4. Mongoose Official Docs

- **Website:** <https://mongoosejs.com/docs/>

5. FreeCodeCamp Articles

- Mongoose + MongoDB Beginner Article
Read here
- MongoDB + Express.js + Node.js full backend tutorial
Full Stack Guide

6. YouTube - Mongoose Crash Courses

- **Code with Mosh** – Mongoose Basics:
[Watch here](#)
- **Traversy Media** – MERN Stack Backend (Mongoose + MongoDB)
[Watch here](#)